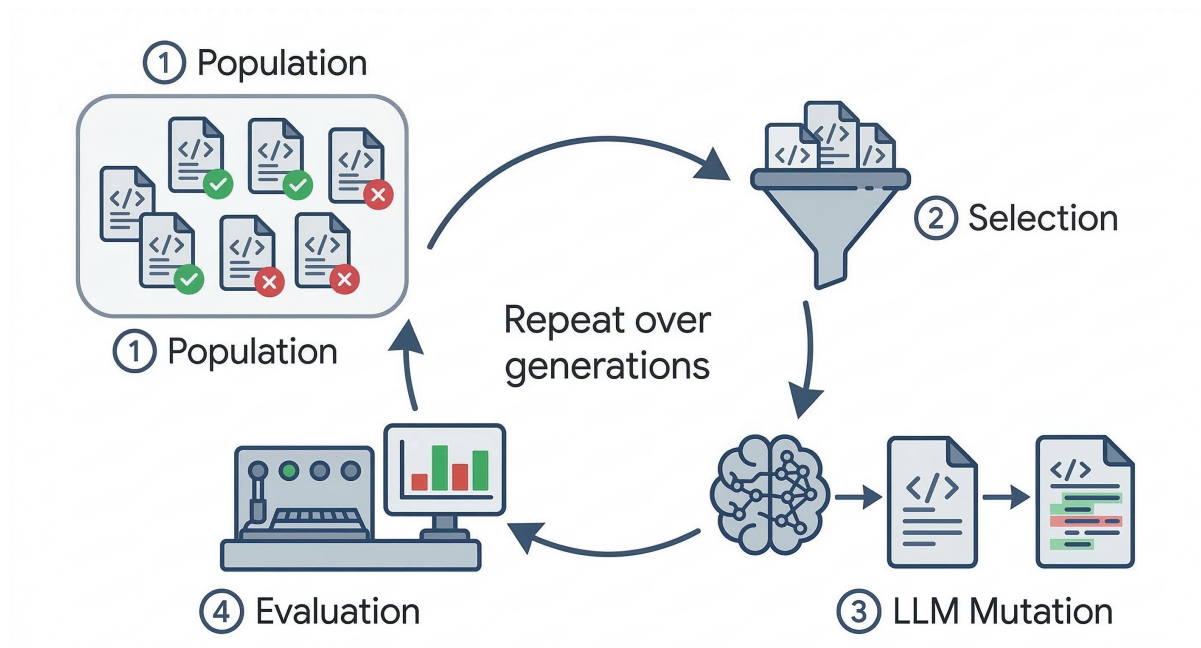


LLM-Driven Evolutionary Systems: A Friendly Introduction

If you're comfortable with programming but haven't encountered genetic or evolutionary algorithms before, this short guide will get you up to speed on how they work in the context of systems like AlphaEvolve, FunSearch, and ShinkaEvolve.

The Big Idea



Rather than asking an LLM to write a program once, these systems maintain a **population** of many candidate programs and improve them over successive **generations**. Roughly: an LLM reads a high-scoring program, reasons about its strategy, and proposes a targeted modification. An automated **evaluation function** runs the modified code and scores it. High-scoring results feed back into the population, driving further refinement. This loop repeats—gradually discovering better and better solutions by iteratively proposing, testing, and keeping what works.

The twist: unlike classical evolutionary algorithms where "mutations" are random perturbations (e.g., bit flips), each change here is a **reasoned code edit** produced by an LLM—the model reads the parent program, identifies a weakness, and proposes a modification with specific intent (e.g., "replace the greedy heuristic with simulated annealing").

Classical EA: Random Mutation

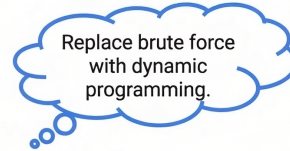


```
def solve(x):  
    return x * x + 1
```

```
def solve(x):  
    return x * x + 5
```

Blind, Random Change

LLM-Driven: Reasoned Mutation



```
def solve(x):  
    return x * x + 1
```

```
def solve(x):  
    memo = {};  
    if x in memo:  
        return memo[x];  
    res = ...;  
    memo[x] = res;  
    return res
```

Intelligent, Strategic Change

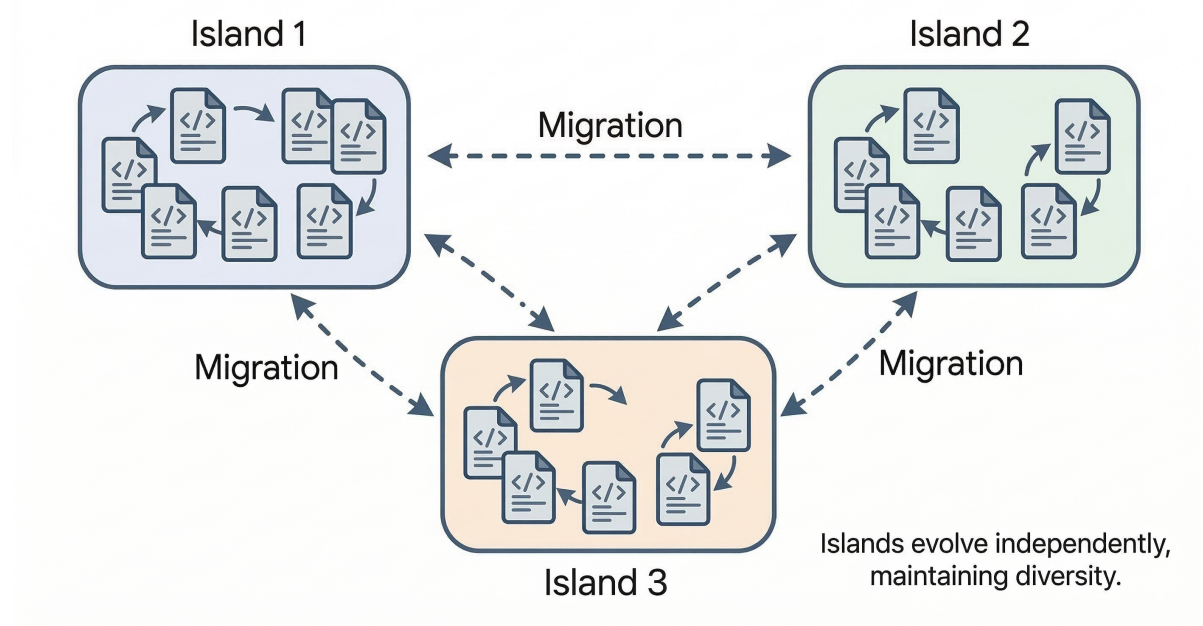
Key Concepts

Fitness function (a.k.a. evaluation function in our systems). This is the function that tells the system how "good" a candidate program is. You provide it—typically as a Python `evaluate` function that runs the program and returns one or more numeric scores. Higher scores mean better solutions. The evolutionary process uses these scores to decide which programs to keep and propagate to future generations. Because it's usually automated (not human judgment), it's often called the *evaluation function* in these systems.

Mutation. A mutation is a change to a single parent program that produces a new candidate. In classical evolutionary algorithms, this might be a random bit flip or subtree swap. In LLM-driven evolution, mutation is an LLM-generated code edit: the model reads the parent, reasons about it, and proposes a targeted modification (often as a search/replace diff). Each mutation is thus a deliberate, semantic change rather than a random perturbation.

Crossover. Crossover is the idea of combining parts of two or more parent solutions to create offspring—like inheriting traits from both parents in genetics. In program evolution, this might mean merging code or ideas from multiple high-scoring programs. Some systems use crossover explicitly; others rely more heavily on mutation.

Island. An "island" is a sub-population that evolves somewhat independently. The overall population is partitioned into several islands; evolution happens within each island, and occasionally individuals may migrate between them. This *island model* helps maintain diversity—different islands can explore different regions of the search space—and avoids premature convergence to a single local optimum. AlphaEvolve's evolutionary database is inspired in part by island-based population models.



Putting It Together

You provide an initial program and an evaluation function. The system starts a population (often seeded with your program or simple variants) and enters an evolutionary loop: the LLM proposes mutations (and sometimes crossover-like combinations), the evaluator scores them, and the best solutions propagate. Islands, when used, let different sub-populations explore in parallel. Over time, the system discovers programs that outperform the initial seed—sometimes dramatically, as seen in results from novel mathematical constructions to algorithmic breakthroughs.

This paradigm has already produced real scientific impact. Understanding these core concepts—evaluation, mutation, crossover, and islands—will help you work with and reason about these systems.